

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-ad526550e33c5192e.jsonl`  
- SHA-256: `fbf7dca7b3b5eaa0e24196c83bd5b4c69431077d15d337d93f39ce23fd598031`  
- Source modified: `2026-07-02T03:00:07+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:58:05.136Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "/api/source re-downloads the ENTIRE remote video from the bucket on every HTTP  
request (each browser Range request) into a fresh, never-deleted temp dir",  
  "category": "critical-fix",  
  "severity": "P1",  
  "file": "backend/main.py",  
  "line": 646,  
  "evidence": "backend/main.py:644-651: get_source calls  
storage.acquire_local_input(rec['filepath'], ...) on EVERY request; storage/__init__.py:89-90:  
for a non-local ref this does `tempfile.mkdtemp(prefix='rally-input-')` + full fetch ? a new dir  
+ full download per call, never removed. Proxy warming is explicitly local-only  
(main.py:655-657), so a remote-source video never gets a proxy. FileResponse under  
fastapi>=0.111 (pyproject.toml:28) serves Range ? a browser scrubbing a <video> issues many  
separate requests to the same URL.",  
  "why_it_matters": "Direct cloud-egress cost leak plus a disk-fill denial-of-service of the  
appliance, reachable through a supported flow (remote-registered assets + P8 annotate-in-UI).",  
  "proposed_fix": "Cache the fetched local copy keyed by video_id (e.g. under <output>/proxies  
or a per-video workspace source/ dir) and reuse it across requests; extend proxy warming to  
remote sources; delete or reuse the scratch dir instead of mkdtemp-per-request.",  
  "effort": "M",  
  "tracked_in": ""  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:58:13.347Z)

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
9cd7598 fix(tests): resolve remaining test signature mismatches from app-factory refactor (#371)
de09ec8 refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7) (#370)
b25fdb8 docs: explain lazy cv2 import in --debug-clip path (Q8) (#361)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance + skip_ai_handoff forward) (#341)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)

3. user (2026-07-02T02:58:13.949Z)

```
600         return os.path.join(output_base(cfg), "proxies")
601
602
603     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604         """Background, best-effort: generate the annotation proxy ONCE so subsequent
/api/source loads
605         serve the lighter, seekable copy. Deduped; never blocks the request; the source
keeps serving if
606         it fails."""
607         with _proxy_lock:
608             if video_id in _proxy_inflight:
609                 return
610             _proxy_inflight.add(video_id)
611         try:
612             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
613             if not needs_proxy(src_local):
614                 return # already light enough ? no proxy worth making
615             make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
f"{video_id}.mp4"))
616         except Exception as e: # noqa: BLE001 - warming is best-effort
617             logger.warning("proxy: warm failed for %s: %s", video_id, e)
618         finally:
619             with _proxy_lock:
620                 _proxy_inflight.discard(video_id)
621
622
623     @app.get("/api/source/{video_id}")
624     def get_source(video_id: str, request: Request):
625         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
626         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
627         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
628         background for next time ? non-blocking, so the first load is never delayed. Proxy
warming is
629         LOCAL-source only (remote-source proxying is a follow-up)."""
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
```

```

634
635     # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636     try:
637         proxy_name = safe_output_filename(f"{video_id}.mp4")
638         proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
639 _proxies_dir(cfg))
640     except ValueError:
641         proxy_path = ""
642     if proxy_path and os.path.isfile(proxy_path):
643         return FileResponse(proxy_path, media_type="video/mp4")
644
645     # 2) Otherwise resolve + serve the source.
646     try:
647         local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
648 None))
649         is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
650 None))
651     except ValueError as e: # unsupported scheme etc. ? a clean client error, never a
652 raise HTTPException(status_code=400, detail=str(e)) from e
653 except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
654 (not swallowed)
655     raise HTTPException(status_code=502, detail=f"Could not resolve the source
656 video: {e}") from e
657     if not os.path.isfile(local):
658         raise HTTPException(status_code=404, detail="Source video file not found.")
659
660     # 3) Warm a proxy in the background for next time (local source only).
661     if is_local:
662         threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
663 daemon=True).start()
664
665     media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
666     return FileResponse(local, media_type=media)
667
668 @app.post("/api/annotations")
669 def post_annotations(req: AnnotationsRequest, request: Request):
670     """Persist finished rally labels as the annotator CSV (P8), written verbatim to
671     ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
672     ``video_id`` ? so it joins
673     the video by content hash exactly as the training pipeline expects. Byte-compatible
674     with the VLC
675     plugin + browser extension. A later step (owner-gated) promotes labels into the
676     durable corpus.
677
678     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
679     validated as a bare
680     filename so an untrusted value can't escape the labels dir (arbitrary write)."""
681     try:
682         edge_auth.resolve_tenant(request.headers, request.app.state.config)
683     except edge_auth.EdgeAuthError as e:
684         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
685
686     cfg = request.app.state.config
687     labels_dir = os.path.join(output_base(cfg), "labels")
688     try:
689         name = safe_output_filename(f"{req.video_id}.rallies.csv")
690     except ValueError as e:
691         raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
692     os.makedirs(labels_dir, exist_ok=True)
693     path = os.path.join(labels_dir, name)
694     try:
695         path = resolve_under_root(path, labels_dir)
696     except ValueError as e:

```

```
688         raise HTTPException(status_code=400, detail=str(e)) from e
689         with open(path, "w", encoding="utf-8", newline="") as f:
```

4. user (2026-07-02T02:58:20.943Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\kheIsutra-guru.

5. user (2026-07-02T02:58:26.436Z)

No files found

6. user (2026-07-02T02:58:31.837Z)

badminton-highlight-indexer\backend\storage__init__.py:72: def acquire_local_input(

7. user (2026-07-02T02:58:36.540Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13
14     from __future__ import annotations
15
16     import os
17     import tempfile
18     from typing import Any, Dict, Iterator, Optional
19
20     from backend.storage.base import StorageBackend
21     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
22
23     __all__ = [
24         "StorageBackend",
25         "StorageRef",
26         "StorageStat",
27         "parse_uri",
28         "resolve_input_uri",
29         "get_storage",
30         "fetch",
31         "put",
32         "list_uris",
33         "acquire_local_input",
34         "input_is_local",
35         "resolve_input_ref",
36     ]
37
38
39     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
40         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
41         if storage_config is None:
42             return {}
```

```

43         if hasattr(storage_config, "model_dump"):
44             return storage_config.model_dump() # type: ignore[no-any-return]
45         return dict(storage_config)
46
47
48     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
49         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
50         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
51         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
52
53         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
54         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
55         sc = _storage_dict(storage_config)
56         uri = resolve_input_uri(
57             raw,
58             input_backend=sc.get("input_backend", "local"),
59             input_root=sc.get("input_root", ""),
60         )
61         return parse_uri(uri)
62
63
64     def input_is_local(raw: str, storage_config: Any = None) -> bool:
65         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
67         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68         ``resolve_input_ref``)."""
69         return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72     def acquire_local_input(
73         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74     ) -> str:
75         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82         worker's ``cmd_infer`` already ships."""
83         sc = _storage_dict(storage_config)
84         ref = resolve_input_ref(raw, sc)
85         if ref.backend == "local":
86             return (
87                 ref.key
88             ) # identity ? no temp, no copy, byte-identical to passing the path directly
89         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91
92
93     def _backend_class(name: str):
94         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""

```

```

95     if name == "local":
96         from backend.storage.local import LocalStorage
97
98         return LocalStorage
99     if name == "s3":
100         from backend.storage.s3 import S3Storage
101
102         return S3Storage
103     if name == "gcs":
104         from backend.storage.gcs import GCSStorage
105
106         return GCSStorage
107     if name == "gdrive":
108         from backend.storage.gdrive import GDriveStorage
109
110         return GDriveStorage
111     if name == "gdrive-api":
112         from backend.storage.gdrive_api import GDriveApiStorage
113
114         return GDriveApiStorage
115     raise ValueError(
116         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
117     )
118
119
120 def get_storage(
121     backend: Optional[str] = None,
122     *,
123     config: Optional[Dict[str, Any]] = None,
124     **overrides,
125 ) -> StorageBackend:
126     """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
127     selects the implementation; its per-backend settings come from ``config[backend]`` merged
128     with ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
129     selectors."""
130     cfg = config or {}
131     name = backend or cfg.get("backend", "local")
132     # Per-backend settings live under the backend name; tolerate the hyphen?underscore
133     # form so a hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
134     # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
135     settings: Dict[str, Any] = dict(
136         cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
137     )
138     settings.update(overrides)
139     return _backend_class(name)(**settings)
140
141 def fetch(
142     uri: str,
143     dst: "str | os.PathLike[str] | None" = None,
144     *,
145     config: Optional[Dict[str, Any]] = None,
146     overwrite: bool = False,
147 ) -> str:
148     """Resolve ``uri`` to a local file path. For ``local`` this is an identity
149     passthrough (no copy); for s3/gcs it downloads to ``dst``."""
150     ref = parse_uri(uri)
151     return get_storage(ref.backend, config=config).fetch_to_local(
152         ref, dst, overwrite=overwrite
153     )

```

```

154
155
156 def put(
157     local_path: "str | os.PathLike[str]",
158     uri: str,
159     *,
160     config: Optional[Dict[str, Any]] = None,
161     content_type: Optional[str] = None,
162 ) -> str:
163     """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
164     ref = parse_uri(uri)
165     get_storage(ref.backend, config=config).put(
166         local_path, ref, content_type=content_type
167     )
168     return uri
169
170
171 def list_uris(
172     prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
173 ) -> Iterator[str]:
174     """Yield the URI of every object under ``prefix_uri``."""
175     ref = parse_uri(prefix_uri)
176     for r in get_storage(ref.backend, config=config).list(ref):
177         yield r.uri
178

```

8. user (2026-07-02T02:58:51.318Z)

```

478: def _friendly_video_name(filepath: str) -> str:
481:     Falls back to the whole ``filepath`` if there's no basename.
482:     base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
484:     return stem or base or str(filepath or "")
501:     """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
`id`/`filepath` (an
508:     fp = row.get("filepath") or ""
631:     if not rec or not rec.get("filepath"):
646:         local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
647:         is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage", None))
855:     if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
1194:         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1196:         db.add_video(
1212:         db.add_video(
1320:         # still preserved in DB provenance via db.add_video.
1603:     {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1622:     # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
1626:     video["filepath"],
1646:     and return the {status, filepath, download_url} result the SPA polls for. Returns a
1680:     # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1684:     video["filepath"], getattr(cfg, "storage", None)
1697:     # `filepath` is server-local (not browser-reachable); also return the download URL so
the SPA needn't
1703:     "filepath": out_path,
2070:     db_instance.add_video(
2125:     # in DB provenance via db.add_video.

```

9. user (2026-07-02T02:58:51.892Z)

```

28:     "fastapi>=0.111.0",
52:     "httpx",          # fastapi TestClient transport

```

10. user (2026-07-02T02:59:00.897Z)

```

820         medium_backend = medium_ref.backend
821         if medium_ref.backend == "local":
822             # "This computer" ? the upload already lives here; it IS the durable copy
823             (no duplicate).
824             stored_uri = os.path.abspath(local_working)
825         else:
826             dest = _asset_dest_uri(req.medium_uri, video_id, ext)
827             # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud bucket is
828             warn-only.
829             local_dir = _medium_local_dir(medium_ref, storage_settings)
830             if local_dir is not None:
831                 require_free_bytes(
832                     local_dir, os.path.getsize(local_working), stage="store"
833                 )
834             else:
835                 logger.info(
836                     "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota warn
837                     only).",
838                     medium_ref.backend,
839                 )
840             try:
841                 stored_uri = storage.put(local_working, dest, config=storage_settings)
842                 copied = True
843             except ValueError as e:
844                 raise HTTPException(status_code=400, detail=str(e)) from e
845             except Exception as e: # noqa: BLE001 ? a put failure is surfaced, never
846                 silently dropped
847                 logger.error(
848                     "[ASSETS] put failed video_id=%s dest=%s: %s",
849                     video_id,
850                     dest,
851                     e,
852                     exc_info=True,
853                 )
854                 raise HTTPException(
855                     status_code=502, detail=f"could not store into the medium: {e}"
856                 ) from e
857
858         # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
859         videos").
860         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
861             raise HTTPException(status_code=500, detail="failed to register the asset.")
862         logger.info(
863             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
864             video_id,
865             medium_backend,
866             stored_uri,
867             copied,
868             owner_id,
869         )
870         return JSONResponse(
871             status_code=201,
872             content={
873                 "video_id": video_id,
874                 "medium": medium_backend,

```

11. user (2026-07-02T02:59:01.561Z)

```

1185         segmenter_actual = None
1186         segmentation_ran = False
1187         # Compute-path PROVENANCE (the validation method): which backend actually ran the
1188         DETECT ?
1189         # "in_process" once the local segmenter is reached, set on the response as
1190         "cloud_run" by
1191         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config

```



```

fail).
1190     compute_actual: Optional[str] = None
1191     response: Optional[Dict[str, Any]] = None
1192     try:
1193         if failures:
1194             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195             # status; it no longer destroys the video's prior good segments.
1196             db.add_video(
1197                 video_id,
1198                 req.video_path,
1199                 "failed",
1200                 [r.model_dump() for r in val_results],
1201             )
1202             response = {
1203                 "video_id": video_id,
1204                 "status": "failed",
1205                 "message": "Video failed validation checks.",
1206                 "results": [r.model_dump() for r in val_results],
1207             }
1208             return response
1209
1210         # 2. Run segmenter & indexer
1211         logger.info("[API] Video passed checks. Segmenting and indexing...")
1212         db.add_video(
1213             video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
1214         )
1215
1216         segmenter_cls = segmenter_registry.get(seg_name)
1217         if not segmenter_cls:
1218             # Submit-time validation already 400s unknown names; this is the defensive
1219             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
1220             available = list(segmenter_registry.list_available().keys())
1221             response = {
1222                 "video_id": video_id,
1223                 "status": "failed",
1224                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
1225                 "results": [r.model_dump() for r in val_results],
1226                 "play_segments": [],
1227             }
1228             return response
1229

```

12. user (2026-07-02T02:59:21.362Z)

```

CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py

```

```
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training
0.136.1 1.0.0
```

13. user (2026-07-02T02:59:21.518Z)

```
badminton-highlight-indexer\backend\storage\__init__.py:89:      scratch =
tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
```

14. user (2026-07-02T03:00:07.142Z)

Structured output provided successfully